

ILCMS Technical Specification v2.2

RAG Data Model, PostgreSQL Schema & API Specification

Icelandic Legal Case Management System (ILCMS)

Developer Database & API Baseline

3 September 2026

Implementation Position

- v2.2 defines the concrete persistence and API layer for the v2.1 RAG architecture.
- Initial implementation uses PostgreSQL + pgvector, with an abstraction boundary for future OpenSearch/vector infrastructure.
- Authorization, provenance and rebuildability are treated as first-class database requirements.

1. Purpose and Scope

- v2.2 defines the concrete PostgreSQL data model, pgvector storage model, constraints, indexes, migration strategy and REST API contracts for the ILCMS RAG subsystem.
- It is the database/API companion to v2.1. The authoritative ILCMS case, party, document, authorization and audit models remain the source of truth.

2. Design Principles

- UUID primary keys are used for distributed-safe identifiers; human-readable case numbers remain separate.
- Foreign keys and database constraints enforce structural integrity; application services enforce domain authorization and complex business rules.
- AI/RAG data is derived except for audit/provenance records and approved AI outputs.
- Document versions are immutable. Re-indexing creates/rebuilds derived records rather than rewriting historical source material.
- All timestamps are timezone-aware; store canonical timestamps in UTC and preserve the relevant ILCMS timezone metadata.
- Use expand/contract migrations for production deployments.

3. PostgreSQL Extensions

- Enable pgvector in the ILCMS database. The exact extension version is pinned by the deployment baseline.
- Use pgvector vector type for embeddings when dimensions are known and compatible with the selected model. pgvector currently supports vector dimensions up to 2,000 for the vector type and provides HNSW and IVFFlat approximate indexes. ■cite■turn0search0■
- Use PostgreSQL full-text search and pg_trgm for lexical retrieval. Vector and lexical retrieval are combined in the application retrieval service.

4. Schema Namespace

- Recommended schemas: public for existing ILCMS domain tables; ai for AI governance/run records; rag for chunks, embeddings and index generations.
- Example: CREATE SCHEMA IF NOT EXISTS ai; CREATE SCHEMA IF NOT EXISTS rag;
- Keep AI/RAG tables logically separated so privileges, migrations and operational ownership can be controlled independently.

5. ai.ai_providers

- Columns: id UUID PK; name VARCHAR(120) UNIQUE NOT NULL; provider_type VARCHAR(40) NOT NULL; endpoint TEXT; deployment_type VARCHAR(30) NOT NULL; data_region VARCHAR(100); retention_policy TEXT; training_policy TEXT; enabled BOOLEAN NOT NULL DEFAULT FALSE; created_at TIMESTAMPTZ NOT NULL; updated_at TIMESTAMPTZ NOT NULL.
- provider_type examples: LOCAL, PRIVATE, EXTERNAL. deployment_type examples: AIR_GAPPED, PRIVATE_NETWORK, EXTERNAL_API.
- External providers require explicit governance approval before enabled=true.

6. ai.ai_models

- Columns: id UUID PK; provider_id UUID FK; name; version; digest; model_type; capabilities JSONB; context_limit INTEGER; dimensions INTEGER NULL; license TEXT; status; created_at; published_at; retired_at.
- model_type: CHAT, EMBEDDING, RERANKER, MULTIMODAL.
- UNIQUE(provider_id, name, version, digest).
- Published model versions are immutable. Digest should identify the exact model artefact where feasible.

7. ai.ai_prompts

- Columns: id UUID PK; prompt_key; version INTEGER; language VARCHAR(10); template TEXT; input_schema JSONB; output_schema JSONB; purpose TEXT; status; owner_user_id; created_at; published_at; retired_at.

- UNIQUE(prompt_key, version).
- Status: DRAFT, EVALUATION, APPROVED, PUBLISHED, RETIRED.
- Never edit a PUBLISHED prompt in place.

8. ai.ai_policies

- Columns: id UUID PK; policy_key; version INTEGER; risk_class; allowed_data_classes JSONB; allowed_models JSONB; external_allowed BOOLEAN; human_approval_required BOOLEAN; max_context_tokens INTEGER; retention_days INTEGER; status; created_at; published_at.
- UNIQUE(policy_key, version).
- Policy evaluation returns allow/deny plus reasons; policy decisions are persisted with the AI run.

9. ai.ai_runs

- Columns: id UUID PK; organization_id UUID NOT NULL; case_id UUID NULL; user_id UUID NOT NULL; use_case VARCHAR(80); risk_class; status; model_id UUID; prompt_id UUID; policy_id UUID; retrieval_config JSONB; request_id VARCHAR(100); idempotency_key VARCHAR(200); input_scope JSONB; usage_metrics JSONB; started_at; completed_at; error_code; created_at.
- Unique idempotency constraint should be scoped to organization/user/use-case according to the API contract.
- Indexes: (organization_id, created_at DESC), (case_id, created_at DESC), (user_id, created_at DESC), (status, created_at).

10. ai.ai_run_sources

- Columns: run_id UUID FK; document_version_id UUID FK; chunk_id UUID FK; rank INTEGER; retrieval_method VARCHAR(30); retrieval_score DOUBLE PRECISION; citation_key VARCHAR(40); created_at.
- PRIMARY KEY(run_id, chunk_id).
- Store the retrieval evidence used to construct context; never rely on a later search to reconstruct historical provenance.

11. ai.ai_run_outputs

- Columns: id UUID PK; run_id UUID FK; output_type VARCHAR(40); content TEXT NULL; structured_json JSONB NULL; validation_status; approval_status; approved_by UUID NULL; approved_at TIMESTAMPTZ NULL; created_at.
- Approval states: NOT_REQUIRED, AWAITING_APPROVAL, APPROVED, REJECTED.
- Approved authoritative data is written through the relevant normal ILCMS service, not by directly updating domain tables from the model.

12. ai.ai_feedback and ai.ai_incidents

- Feedback: id, run_id, user_id, rating, category, comment, created_at.
- Incident: id, run_id, category, severity, description, status, owner_user_id, resolution, created_at, resolved_at.
- Incidents must be auditable and capable of disabling a model, provider, prompt or use case through administration controls.

13. rag.rag_index_generations

- Columns: id UUID PK; name; corpus_scope JSONB; embedding_model_id UUID; embedding_dimensions INTEGER; distance_metric VARCHAR(20); chunking_version VARCHAR(40); tokenizer_version VARCHAR(80); status; document_count BIGINT; chunk_count BIGINT; created_at; activated_at; retired_at.
- Status: BUILDING, VALIDATING, ACTIVE, RETIRED, FAILED.
- Only an explicitly activated generation is used for production retrieval.

14. rag.document_chunks

- Columns: id UUID PK; document_version_id UUID NOT NULL; case_id UUID NOT NULL; organization_id UUID NOT NULL; index_generation_id UUID NOT NULL; chunk_index INTEGER NOT NULL; content TEXT NOT NULL; content_hash CHAR(64) NOT NULL; page_start INTEGER; page_end INTEGER; section_path TEXT; token_count INTEGER; language VARCHAR(10); classification VARCHAR(50); acl_snapshot_hash CHAR(64); metadata JSONB; created_at.
- UNIQUE(document_version_id, index_generation_id, chunk_index).
- Index content with PostgreSQL FTS and metadata with appropriate B-tree/GIN indexes.

15. rag.document_embeddings

- Columns: id UUID PK; chunk_id UUID NOT NULL; embedding_model_id UUID NOT NULL; dimensions INTEGER NOT NULL; embedding VECTOR(N); content_hash CHAR(64) NOT NULL; created_at.
- UNIQUE(chunk_id, embedding_model_id, content_hash).
- N is fixed per deployed embedding model/index. If multiple dimensional models are simultaneously required, use separate embedding columns/tables or model-specific tables rather than a polymorphic vector dimension.

16. RAG Indexing SQL

- Illustrative setup: CREATE EXTENSION IF NOT EXISTS vector; CREATE INDEX ... USING hnsu (embedding vector_cosine_ops); CREATE INDEX ... USING gin (to_tsvector('simple', content));
- pgvector documents exact search as the default and HNSW/IVFFlat as approximate alternatives. HNSW generally offers a stronger speed/recall tradeoff but requires more memory and slower builds; tune using measured workload data. ■cite■turn0search0■
- Do not create an approximate index until the workload and filtering behaviour have been benchmarked.

17. Authorization Metadata

- Each chunk includes organization_id, case_id and an ACL snapshot hash. Additional ACL material may be represented in metadata or a normalized rag_chunk_acl table if required.
- For highly restricted installations, prefer a normalized ACL relation: rag.chunk_acl(chunk_id, principal_type, principal_id, access_level).
- Application authorization remains authoritative. The index is an acceleration mechanism and security filter, not the policy engine.

18. Multi-Tenant Isolation

- Every RAG query must include organization scope before vector retrieval.
- For large multi-tenant deployments, consider partitioning by organization or another security boundary. pgvector documentation notes that shared approximate indexes can affect recall across tenants and recommends partitioning or separate tables for tenant isolation. ■cite■turn0search0■
- Do not rely on application-supplied organization_id alone; derive tenant context from authenticated identity and authorized membership.

19. Search Indexes

- Recommended indexes: document_chunks(case_id); document_chunks(organization_id, case_id); document_chunks(document_version_id); document_chunks(content_hash); document_chunks(index_generation_id); GIN/FTS index on content; GIN index on metadata where useful.
- Use partial or composite indexes where confidentiality/status filters are highly selective.
- Benchmark index sizes and query plans before adding broad JSONB indexes.

20. RAG Query Transaction

1. Begin read-only transaction where practical.
2. Resolve authenticated user and organization.
3. Authorize case/document scope.
4. Load active index generation.
5. Generate query embedding.
6. Execute lexical/vector retrieval with ACL filters.
7. Fuse/rerank candidates.
8. Persist ai_run and ai_run_sources.
9. Build context from recorded sources.
10. Call model gateway outside long-lived DB locks.
11. Persist output and audit event transactionally.

21. Hybrid Search API

- POST /api/v1/ai/search
- Request: {query, case_id?, document_ids?, language?, top_k?, filters?, use_case}.

- Response: {run_id, results:[{citation_key, document_id, document_version_id, page_start, page_end, title, snippet}], index_generation, retrieval_methods}.
- Search results must be authorization-filtered before returning to the client.

22. RAG Chat API

- POST /api/v1/ai/chat
- Request: {case_id, message, document_ids?, language?, conversation_id?, use_case}.
- Response: {run_id, status, answer, citations[], warnings[], model_version, prompt_version, policy_version}.
- Use HTTP 202 for asynchronous generation; GET /api/v1/ai/runs/{run_id} returns progress/result.

23. Source API

- GET /api/v1/ai/sources/{citation_key}
- Resolve citation to authorized document/version/page/section and return navigation metadata.
- Do not expose object-storage keys or internal paths. Use the normal document service/signed URL mechanism.

24. Administration APIs

- GET/POST /api/v1/admin/ai/providers
- GET/POST /api/v1/admin/ai/models
- GET/POST /api/v1/admin/ai/prompts
- GET/POST /api/v1/admin/ai/policies
- GET/POST /api/v1/admin/ai/index-generations
- GET /api/v1/admin/ai/runs
- GET/POST /api/v1/admin/ai/incidents
- Administrative APIs require explicit AI administration permissions and must not automatically grant access to confidential case content.

25. OpenAPI Requirements

- All endpoints are included in the generated FastAPI OpenAPI document.
- Define request/response Pydantic models independently from SQLAlchemy models.
- Use enums for status/risk/type fields.
- Document 401, 403, 404, 409, 422 and 429 responses.
- Include request_id and run_id in error/async responses where applicable.

26. Pagination and Filtering

- Admin/run listing uses page/page_size initially, maximum 100, matching Version 1 conventions.
- Filters and sorts are allowlisted, never converted directly into SQL fragments.
- Future high-volume audit/run APIs may add cursor pagination.

27. Concurrency and Idempotency

- Use Idempotency-Key for POST operations that create AI runs or indexing jobs.
- Use optimistic concurrency/version checks for administrative model/prompt/policy publication.
- Index workers use content hashes and unique constraints to make retries safe.
- AI run completion must be atomic from the application's perspective: output state, provenance and required audit event are committed together.

28. Migration Strategy

- Migration 001: schemas/extensions and provider/model registries.
- Migration 002: prompt/policy/run/audit tables.
- Migration 003: index generations/chunks/embeddings.

- Migration 004: indexes and constraints after representative data exists where appropriate.
- Production migration uses expand → backfill → validate → switch → contract. Avoid destructive changes in the same release as application changes that still depend on the old schema.

29. Backup and Recovery

- Back up AI governance/run/audit tables according to the ILCMS database policy.
- RAG chunks and embeddings are rebuildable from authoritative document versions, so they can be treated as derived data; however, retaining them can reduce recovery time.
- After restore, verify active index generation consistency and schedule a controlled index integrity/rebuild check.

30. Performance Targets

- Initial engineering targets should be measured rather than treated as contractual guarantees: lexical/vector candidate retrieval under a few hundred milliseconds on the target corpus; API orchestration overhead kept small relative to model latency.
- Benchmark exact search against HNSW. pgvector recommends comparing approximate retrieval with exact search to monitor recall.
■cite■turn0search0■
- Record p50/p95 retrieval latency, candidate counts, reranking latency and database buffer/cache behaviour.

31. Security Requirements

- Least-privilege DB roles: application read/write, migration owner, indexing worker and read-only reporting roles should be separated where practical.
- Never expose database credentials to model servers.
- Never let the model construct arbitrary SQL.
- RAG query parameters are validated and bounded.
- Sensitive source content is not placed in ordinary application logs.
- Row-level security may be used as defense in depth, but it must not replace application authorization.

32. API Error Model

- Use the Version 1 standard error object: {error:{code,message,request_id}}.
- Suggested AI codes: AI_POLICY_DENIED, AI_MODEL_UNAVAILABLE, AI_PROVIDER_DISABLED, AI_CONTEXT_LIMIT, AI_RETRIEVAL_FAILED, AI_CITATION_INVALID, AI_APPROVAL_REQUIRED, AI_RUN_NOT_FOUND, AI_INDEX_NOT_READY, AI_IDEMPOTENCY_CONFLICT.

33. Initial SQL Skeleton

- CREATE EXTENSION IF NOT EXISTS vector;
- CREATE SCHEMA IF NOT EXISTS ai;
- CREATE SCHEMA IF NOT EXISTS rag;
- CREATE TABLE rag.document_chunks (id uuid PRIMARY KEY, document_version_id uuid NOT NULL, case_id uuid NOT NULL, organization_id uuid NOT NULL, index_generation_id uuid NOT NULL, chunk_index integer NOT NULL, content text NOT NULL, content_hash char(64) NOT NULL, page_start integer, page_end integer, section_path text, token_count integer, language varchar(10), classification varchar(50), acl_snapshot_hash char(64) NOT NULL, metadata jsonb, created_at timestamptz NOT NULL DEFAULT now(), UNIQUE(document_version_id,index_generation_id,chunk_index));
- CREATE TABLE rag.document_embeddings (id uuid PRIMARY KEY, chunk_id uuid NOT NULL REFERENCES rag.document_chunks(id), embedding_model_id uuid NOT NULL, dimensions integer NOT NULL, embedding vector(1536) NOT NULL, content_hash char(64) NOT NULL, created_at timestamptz NOT NULL DEFAULT now(), UNIQUE(chunk_id,embedding_model_id,content_hash));
- The 1536 dimension is illustrative only. Production schema must use the actual approved embedding model dimension.

34. SQLAlchemy Mapping Guidance

- Use UUID types, timezone-aware DateTime, JSONB and pgvector's Vector type through the selected SQLAlchemy integration.
- Do not put retrieval authorization logic in SQLAlchemy model methods.
- Repositories expose typed operations such as get_active_index(), search_lexical(), search_vector(), save_run(), save_sources().
- Services coordinate authorization and transactions.

35. Acceptance Tests

- Create two organisations and verify vector queries cannot cross organisation boundaries.
- Create two cases in one organisation with disjoint ACLs and verify semantic search cannot cross case access.
- Remove a user's case permission and verify subsequent queries fail or omit the source.
- Change document version and verify old chunks remain traceable while new index generation can coexist.
- Repeat an indexing job and verify no duplicate chunks/embeddings are created.
- Repeat an AI request with the same Idempotency-Key and verify one logical run.
- Attempt fabricated citation key and verify API/output validation rejects it.
- Disable provider and verify core ILCMS APIs continue operating.

36. Recommended Build Order

- 1. Schemas/extensions.
- 2. AI governance tables.
- 3. RAG index/chunk tables.
- 4. Embedding storage and indexes.
- 5. Authorization-aware retrieval repository.
- 6. Hybrid search service.
- 7. AI run/provenance persistence.
- 8. Chat API.
- 9. Citation resolver.
- 10. Administration APIs.
- 11. Migration/backup/restore tests.
- 12. Load/security/regression testing.

37. Final Engineering Position

- PostgreSQL remains the authoritative transactional platform. pgvector is an integrated retrieval capability, not a second source of truth. This minimises operational complexity while leaving an abstraction boundary for a future dedicated search/vector platform.
- The database must make secure behaviour easy: tenant/case identifiers, provenance, immutable version references, content hashes, explicit index generations and strong uniqueness constraints are part of the RAG security model, not merely implementation details.

Appendix A — Entity Summary

- AI governance: ai_providers, ai_models, ai_prompts, ai_policies.
- Execution/provenance: ai_runs, ai_run_sources, ai_run_outputs, ai_feedback, ai_incidents.
- RAG: rag_index_generations, document_chunks, document_embeddings.
- Authoritative source remains Version 1 document/case data.

Appendix B — Non-Negotiable Rules

1. Never retrieve before authorization.
2. Never use vector similarity as an authorization decision.
3. Never overwrite immutable document versions.
4. Never make embeddings authoritative.
5. Never allow arbitrary SQL from the model.
6. Never expose restricted source content through citations.
7. Never change a published model/prompt/policy version in place.
8. Never make external AI connectivity mandatory.